

4. Ensayos y resultados	35
4.1. Ensayos sobre la aplicación	35
4.2. Ensayos sobre el sistema	36
4.3. Ensayos de laboratorio	38
5. Conclusiones	43
5.1. Conclusiones generales	43
5.2. Próximos pasos	44
Bibliografía	45

4. Ensayos y resultados	35
4.1. Ensayos sobre la aplicación	35
4.2. Ensayos sobre el sistema	36
4.3. Ensayos de laboratorio	39
5. Conclusiones	43
5.1. Conclusiones generales	43
5.2. Próximos pasos	44
Bibliografía	45

Índice de figuras

1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control.	3
2.1. Arquitectura general del sistema.	7
2.2. Microcontrolador ESP32 ¹	10
2.3. Funcionamiento del módulo transceptor.	10
2.4. Módulo W5100 ²	11
2.5. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio.	13
3.1. Esquema de interconexión entre el ESP32 y los módulos externos.	15
3.2. Módulo transceptor comercial perteneciente a Enertik.	16
3.3. Diagrama de cómo se establece la comunicación CAN entre la red del motorhome y el ESP32.	17
3.4. Diseño de divisor resistivo para adaptación UART.	17
3.5. Prototipo experimental del equipo.	19
3.6. Diseño final para PCB del equipo.	21
3.7. Diagrama final de los componentes de hardware empleados.	22
3.8. Relación entre las distintas tareas del firmware y su comunicación con el exterior.	23
3.9. Contenedores dentro de una pantalla.	28
3.10. Pestaña principal de la aplicación.	28
3.11. Pestaña luces de la aplicación.	30
3.12. Pestaña motores de la aplicación.	31
3.13. Pestaña energía de la aplicación.	32
4.1. Entorno de ejecución y depuración de la aplicación.	35
4.2. Archivo JSON recibido en la pestaña home.	37
4.3. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.	37
4.4. Respuesta del servidor ante una solicitud de control.	38
4.5. Módulo PRC10 utilizado en los ensayos.	38
4.6. Equipos Enertik utilizados en los ensayos.	39
4.7. Equipo Victron MultiPlus-II y módulo Cerbo GX.	40

Índice de figuras

1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control.	3
2.1. Arquitectura general del sistema.	7
2.2. Microcontrolador ESP32 ¹	10
2.3. Funcionamiento del módulo transceptor.	10
2.4. Módulo W5100 ²	11
2.5. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio.	13
3.1. Esquema de interconexión entre el ESP32 y los módulos externos.	15
3.2. Módulo transceptor comercial perteneciente a Enertik.	16
3.3. Diagrama de cómo se establece la comunicación CAN entre la red del motorhome y el ESP32.	17
3.4. Diseño de divisor resistivo para adaptación UART.	17
3.5. Prototipo experimental del equipo.	19
3.6. Diseño final para PCB del equipo.	21
3.7. Diagrama final de los componentes de hardware empleados.	22
3.8. Relación entre las distintas tareas del firmware y su comunicación con el exterior.	23
3.9. Contenedores dentro de una pantalla.	28
3.10. Pestaña principal de la aplicación.	28
3.11. Pestaña luces de la aplicación.	30
3.12. Pestaña motores de la aplicación.	31
3.13. Pestaña energía de la aplicación.	32
4.1. Uso del navegador web como herramienta de depuración.	35
4.2. Archivo JSON recibido en la pestaña home.	37
4.3. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.	38
4.4. Respuesta del servidor ante una solicitud de control.	38
4.5. Módulo PRC10 usado en los ensayos.	39
4.6. Equipos Enertik usados en los ensayos.	40
4.7. Equipo Victron MultiPlus-II 12/3000/120-32 230V y módulo Cerbo GX.	41

Capítulo 2

Introducción específica

En este capítulo se describen los distintos protocolos de comunicación utilizados en el desarrollo del trabajo, los componentes de hardware empleados para cumplir con los objetivos planteados y las herramientas requeridas para el desarrollo del software.

2.1. Arquitectura general del sistema

La figura 2.1 presenta un esquema general del sistema desarrollado, en el que se ilustran los distintos subsistemas involucrados y sus interconexiones.

Como se puede observar, el dispositivo basado en ESP32 actúa como nodo central del sistema y establece comunicación con la red de domótica de Enertik mediante CAN (Controller Area Network) [6], con el ecosistema Victron a través de Modbus TCP sobre Ethernet [7], y con la aplicación móvil mediante solicitudes HTTP (HyperText Transfer Protocol) [8] sobre Wi-Fi.

Esta arquitectura permite integrar subsistemas previamente independientes dentro de una única plataforma de control.

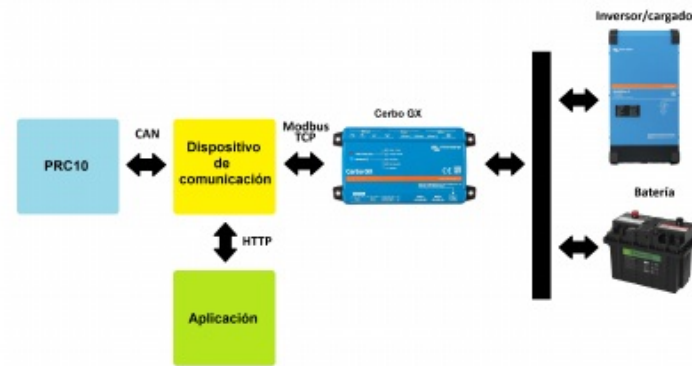


FIGURA 2.1. Arquitectura general del sistema.

Capítulo 2

Introducción específica

En este capítulo se describen los distintos protocolos de comunicación utilizados en el desarrollo del trabajo, los componentes de hardware empleados para cumplir con los objetivos planteados y las herramientas requeridas para el desarrollo del software.

2.1. Arquitectura general del sistema

La figura 2.1 presenta un esquema general del sistema desarrollado, en el que se ilustran los distintos subsistemas involucrados y sus interconexiones.

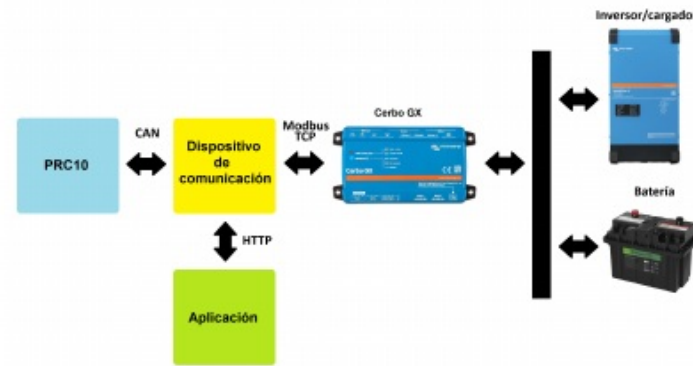


FIGURA 2.1. Arquitectura general del sistema.

Como se puede observar, el dispositivo basado en ESP32 actúa como nodo central del sistema y establece comunicación con la red de domótica de Enertik mediante CAN (Controller Area Network) [6], con el ecosistema Victron a través de Modbus TCP sobre Ethernet [7], y con la aplicación móvil mediante solicitudes HTTP (HyperText Transfer Protocol) [8] sobre Wi-Fi.

Esta arquitectura permite integrar subsistemas previamente independientes dentro de una única plataforma de control.

Capítulo 4

Ensayos y resultados

En este capítulo se explica cómo fueron los ensayos que se realizaron sobre el sistema y qué resultados se obtuvieron a partir de ellos.

4.1. Ensayos sobre la aplicación

En esta etapa se realizaron ensayos sobre la aplicación móvil de forma independiente, con el objetivo de validar su funcionamiento antes de su integración con el sistema embebido.

Para ello, se utilizó el entorno de desarrollo provisto por Ionic mediante el comando `ionic serve`, que permite ejecutar la aplicación en un navegador web. Este entorno facilitó el acceso a herramientas de depuración, tales como la simulación de distintos tamaños de pantalla y la visualización de mensajes en la consola.

La figura 4.1 muestra el entorno de ejecución utilizado durante los ensayos.



FIGURA 4.1. Entorno de ejecución y depuración de la aplicación.

En el sector identificado como cuadro 1, se empleó la funcionalidad de simulación de distintos tamaños de pantalla, lo que permitió evaluar el comportamiento de la interfaz frente a diferentes resoluciones.

Capítulo 4

Ensayos y resultados

En este capítulo se explica cómo fueron los ensayos que se realizaron sobre el sistema y qué resultados se obtuvieron a partir de ellos.

4.1. Ensayos sobre la aplicación

Como se mencionó en la sección 3.3.1, una de las ventajas que aportó el desarrollo de la aplicación mediante Ionic fue la posibilidad de utilizar el navegador web de la computadora como herramienta de depuración.

Ionic dispone del comando `ionic serve`, que permite ejecutar la aplicación en el navegador. A partir de este punto, la aplicación se pone en funcionamiento y puede accederse a ella mediante la dirección localhost y un puerto asociado, que por defecto corresponde al 8100 (aunque este valor puede modificarse según la configuración del entorno de desarrollo).

La figura 4.1 muestra cómo se llevó a cabo este proceso de depuración.

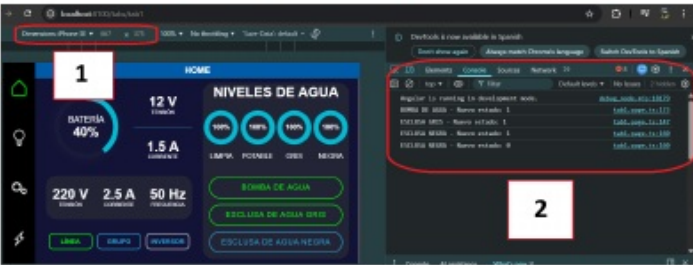


FIGURA 4.1. Uso del navegador web como herramienta de depuración.

Una vez que la aplicación se encuentra en ejecución, es posible acceder a las herramientas de depuración del navegador mediante la tecla F12.

En la figura 4.1 se destacan dos aspectos relevantes al utilizar esta funcionalidad.

En primer lugar, en el sector identificado como cuadro 1, se dispone de una herramienta que permite simular diferentes resoluciones o tamaños de pantalla, lo que facilita la verificación del comportamiento de la aplicación frente a distintos dispositivos.

Por otro lado, en el cuadro 2 se encuentra la consola del navegador, donde se visualizaron mensajes de depuración generados mediante instrucciones como `console.log()`. Estos mensajes incluyeron valores de variables y estructuras de datos en formato `JSON`, lo que facilitó el seguimiento del estado interno de la aplicación y la detección de errores durante su ejecución.

El procedimiento de ensayo consistió en navegar entre las distintas pestañas de la aplicación, accionar los controles disponibles y observar el comportamiento de la interfaz. En paralelo, se analizaron los mensajes generados en la consola del navegador mediante instrucciones de depuración incorporadas en el código.

Adicionalmente, se implementó un esquema de prueba basado en la modificación manual de variables internas (*hardcodeo*), con el fin de simular distintos estados del sistema. Esto permitió evaluar la forma en que la aplicación representa en pantalla valores correspondientes a diferentes condiciones de operación.

Como resultado, se verificó que la navegación entre pantallas se realizaba correctamente, que los controles respondían a las acciones del usuario y que los indicadores visuales reflejaban de manera coherente los valores asignados durante las pruebas.

Asimismo, se comprobó que la interfaz mantenía un comportamiento consistente frente a distintos tamaños de pantalla, sin presentar errores de visualización ni desajustes en la distribución de los elementos.

A partir de estos ensayos, se concluyó que la aplicación presentaba un funcionamiento adecuado a nivel de interfaz y lógica interna, lo que permitió avanzar hacia su integración con el sistema basado en ESP32.

4.2. Ensayos sobre el sistema

En esta etapa se realizaron ensayos de integración entre la aplicación móvil y el sistema basado en ESP32, con el objetivo de validar la comunicación entre ambos y el correcto funcionamiento del servidor implementado en el microcontrolador.

Para ello, se utilizó el entorno de ejecución de la aplicación en el navegador web, junto con el monitor serie del IDE Arduino, empleado para visualizar mensajes de depuración generados por el firmware del ESP32.

El montaje de ensayo consistió en conectar el ESP32 a una red Wi-Fi local, compartida con la computadora en la que se ejecutaba la aplicación.

Al establecer la conexión, el microcontrolador informó su dirección IP a través del monitor serie, lo que permitió acceder a los endpoints del servidor desde la aplicación.

El procedimiento de prueba incluyó el acceso a los distintos endpoints mediante solicitudes HTTP de tipo GET, así como el envío de comandos a través de solicitudes POST generadas desde la interfaz.

Con el fin de evaluar la comunicación, se implementó en el firmware una función temporal que asignaba valores aleatorios a las variables contenidas en las estructuras `JSON`. Estos valores eran informados mediante mensajes en el monitor serie.

Para el desarrollo de este trabajo, se implementaron distintas estrategias de redimensionamiento automático de los elementos gráficos, con el objetivo de lograr una correcta adaptación a diferentes tamaños de pantalla.

Asimismo, el entorno ofrece tamaños predefinidos que corresponden a estándares de diversos modelos de teléfonos móviles. Esta funcionalidad resultó de gran utilidad, ya que permitió evaluar el aspecto de la aplicación en dispositivos representativos sin necesidad de contar físicamente con cada uno de ellos.

Luego, la segunda herramienta de gran utilidad fue la consola del navegador, indicada en el cuadro 2.

En esta consola es posible visualizar mensajes de depuración generados desde la aplicación mediante el uso de instrucciones como `console.log()` dentro del código. Estos mensajes pueden incluir texto, valores de variables y estructuras de datos más complejas, como objetos en formato `JSON` utilizados en la comunicación con el servidor del ESP32.

Asimismo, la consola permite detectar errores durante la ejecución de la aplicación, ya que cualquier falla se reporta mediante mensajes específicos, lo que facilitó su identificación y análisis.

En esta primera etapa de ensayos, se evaluó el funcionamiento de la aplicación de forma independiente. Para ello, se navegó entre las distintas pestañas disponibles, se accionaron los diferentes pulsadores y se analizaron los mensajes generados en la consola.

Adicionalmente, se implementó un esquema de prueba mediante modificación manual de variables internas del código (*hardcodeo*). Esto permitió verificar el comportamiento de los indicadores visuales y evaluar cómo la aplicación representaba en pantalla distintos valores de las magnitudes involucradas.

4.2. Ensayos sobre el sistema

La siguiente etapa consistió en la realización de ensayos en conjunto entre la aplicación y el dispositivo basado en ESP32.

Para ello, además del uso del navegador web como herramienta de depuración de la aplicación, se utilizó el IDE Arduino para analizar el comportamiento del firmware del ESP32.

Este entorno dispone de un monitor serie que permite establecer una comunicación a través de UART. Mediante esta herramienta es posible enviar y recibir datos; sin embargo, en este caso se utilizó principalmente como medio de visualización de mensajes generados por el microcontrolador.

El IDE Arduino incorpora la función `Serial.print()`, que permite la impresión de información en el monitor serie. A través de esta ella se pueden mostrar mensajes de texto, valores de variables y estructuras de datos más complejas, lo que facilitó el seguimiento del estado interno del sistema durante su ejecución.

Para poner a prueba el servidor del ESP32, el microcontrolador se conectó a una red Wi-Fi local, coincidente con la utilizada por la computadora en la que se ejecutaba la aplicación.

A partir de este esquema, se compararon los valores reportados por el ESP32 con los recibidos por la aplicación, y se verificó su coincidencia junto con su correcta representación en la interfaz.

La figura 4.2 muestra un ejemplo del archivo JSON recibido desde el endpoint home, que contiene tanto variables del sistema como estados de los pulsadores.



FIGURA 4.2. Archivo JSON recibido en la pestaña home.

Por otra parte, se evaluó el envío de comandos desde la aplicación hacia el ESP32 mediante solicitudes POST. La figura 4.3 muestra el mensaje recibido por el microcontrolador al accionar el pulsador correspondiente a la bomba de agua.

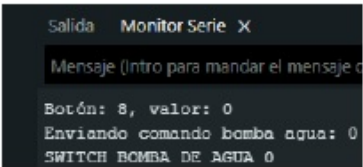


FIGURA 4.3. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.

En este caso, el identificador del comando y el valor asociado fueron correctamente interpretados por el sistema, lo que dio lugar al envío de la instrucción correspondiente hacia el módulo PRC10.

Asimismo, se verificó la respuesta del servidor ante estas solicitudes. La figura 4.4 presenta el contenido de la respuesta enviada por el ESP32, donde se confirma el comando ejecutado junto con el nuevo estado asignado.

Como resultado, se comprobó que los datos intercambiados entre la aplicación y el ESP32 coincidían de manera consistente, que los comandos enviados eran correctamente procesados y que las respuestas del servidor reflejaban el estado actualizado del sistema.

Al momento de establecer la conexión, el ESP32 informó a través del monitor serie la dirección IP asignada por el módem de la red.

A partir de esta dirección IP, y al encontrarse ambos dispositivos en la misma red, fue posible acceder a cada uno de los endpoints del servidor implementado en el ESP32, lo que permitió evaluar su funcionamiento y verificar la correcta comunicación con la aplicación.

Para poner a prueba la comunicación entre los dispositivos, se implementó en el firmware del ESP32 una función temporal en la que se asignaban valores aleatorios a las variables contenidas en las estructuras JSON. Cada vez que uno de estos valores era modificado, se informaba mediante un mensaje en el monitor serie.

Por otro lado, a través de la consola del navegador se verificó que los valores recibidos por la aplicación coincidieran con los reportados en el IDE Arduino. Asimismo, se comprobó que dichos valores se representarían correctamente en la interfaz de la aplicación.

En la figura 4.2 se puede apreciar el comportamiento de la aplicación durante la comunicación con el ESP32.



FIGURA 4.2. Archivo JSON recibido en la pestaña home.

En dicha figura se observa el archivo JSON recibido desde el endpoint home del servidor implementado en el ESP32. Este paquete contiene los valores de todas las variables que la aplicación debe representar en esta pantalla.

Asimismo, se incluyen los estados de los pulsadores correspondientes a la bomba de agua, la esclusa de agua gris y la esclusa de agua negra.

Por otra parte, tal como se mencionó en la sección 3.3.1, al accionar un pulsador la aplicación envía una solicitud HTTP de tipo POST hacia el ESP32. En esta solicitud se indica el origen del comando y el nuevo valor que debe establecerse.

La figura 4.3 muestra el mensaje recibido por el ESP32 al momento de accionar el pulsador de la bomba de agua.

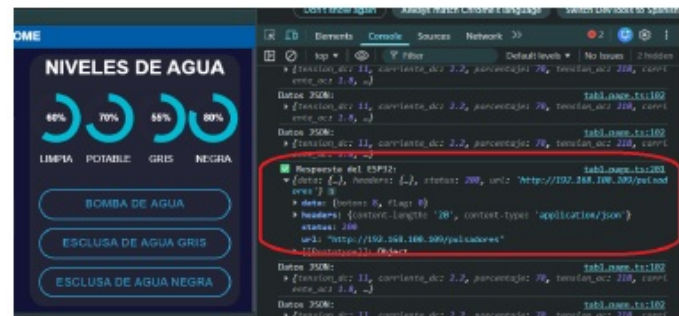


FIGURA 4.4. Respuesta del servidor ante una solicitud de control.

A partir de estos ensayos, se concluyó que la comunicación entre la aplicación y el microcontrolador resultó confiable, lo que permitió validar el funcionamiento del servidor y avanzar hacia la etapa de pruebas con equipos reales.

4.3. Ensayos de laboratorio

En esta etapa se realizaron ensayos con equipos reales de la red de domótica de Enertik, con el objetivo de validar el funcionamiento del sistema en condiciones representativas de operación.

La figura 4.5 muestra el módulo PRC10 utilizado durante los ensayos. Este equipo fue separado y destinado específicamente a este desarrollo.



FIGURA 4.5. Módulo PRC10 utilizado en los ensayos.

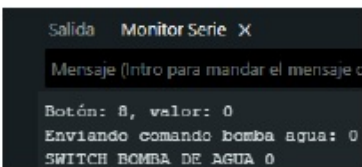


FIGURA 4.3. Mensaje recibido por el ESP32 al accionar un pulsador desde la aplicación.

En este caso, el valor 8 corresponde a un identificador interno del firmware, asociado al pulsador de la bomba de agua. El campo de valor indica el nuevo estado que debe establecerse; en esta solicitud, el valor 0 representa el estado apagado.

En la misma figura también se observa el mensaje “Enviando comando bomba de agua”, que actúa como registro de depuración e indica que la tarea de comunicación CAN envió la orden al módulo PRC10 para ejecutar la acción correspondiente.

Por último, la aplicación, al realizar una solicitud HTTP, espera una respuesta por parte del servidor. La figura 4.4 muestra la respuesta enviada por el ESP32.

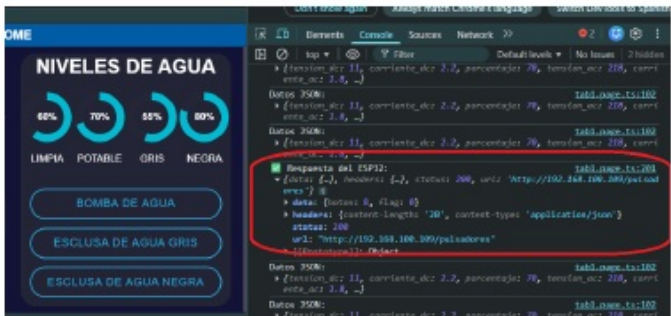


FIGURA 4.4. Respuesta del servidor ante una solicitud de control.

En esta imagen se observa el endpoint utilizado en la consulta junto con el contenido de la respuesta. En particular, se devuelve el comando 8, asociado al pulsador de la bomba de agua, junto con el nuevo valor asignado, lo que confirma la correcta ejecución de la acción.

De esta manera, se evaluó el funcionamiento del servidor implementado en el ESP32 mediante la puesta a prueba de todos los endpoints. Para ello, se asignaron valores aleatorios a las estructuras JSON asociadas a cada uno de ellos, lo que permitió analizar cómo la aplicación representaba dichas respuestas en pantalla.

Asimismo, se verificó el correcto funcionamiento de los pulsadores de la aplicación, así como el envío de comandos y su correspondiente procesamiento en el sistema.

Para la realización de las pruebas, se utilizó un maletín provisto por Enertik que integra los equipos comerciales empleados en aplicaciones de motorhomes. Este conjunto reproduce la configuración real del sistema, tanto a nivel de alimentación como de comunicación.

La figura 4.6 muestra dicho entorno, donde los dispositivos se encuentran interconectados mediante un cableado de cuatro conductores. En ella se identifican el prototipo desarrollado (cuadro 1) y el módulo PRC10 (cuadro 2).



FIGURA 4.6. Equipos Enertik utilizados en los ensayos.

El sistema dispone de dos entradas de alimentación: una fuente de 12 V en corriente continua, que representa la batería, y una entrada de corriente alterna que simula las fuentes de línea, grupo electrógeno e inversor. Como carga, se utilizaron luces testigo que permiten visualizar el accionamiento de los distintos circuitos.

Para la integración con el ecosistema Victron, se emplearon un módulo Cerbo GX y un equipo MultiPlus-II 12/3000/120-32 230V [37], tal como se muestra en la figura 4.7.

El procedimiento de ensayo consistió en instalar la aplicación en un dispositivo móvil con sistema operativo Android, establecer la conexión a la red Wi-Fi generada por el ESP32 y evaluar el comportamiento de cada una de las pestañas.

En conjunto, estos ensayos permitieron validar la comunicación entre la aplicación y el ESP32, así como la correcta interpretación y representación de los datos en la interfaz de usuario, antes de avanzar hacia los ensayos con los equipos de la red de domótica.

4.3. Ensayos de laboratorio

La siguiente etapa consistió en la realización de ensayos con equipos reales de la red de domótica de Enertik.

La figura 4.5 muestra el módulo PRC10 usado durante los ensayos. Este equipo fue separado y utilizado específicamente para el desarrollo de este trabajo.



FIGURA 4.5. Módulo PRC10 usado en los ensayos.

Para poder ensayar una red de domótica completa, se utilizó un maletín proporcionado por Enertik, que integra los equipos comerciales empleados en aplicaciones de motorhomes.

En este maletín, mostrado en la figura 4.6, todos los equipos se encuentran interconectados mediante un cableado de cuatro conductores, que replica la configuración utilizada en los motorhomes.

A su vez, se puede observar el módulo PRC10 usado en los ensayos (indicado con el cuadro 2) junto con el prototipo experimental (indicado con el cuadro 1).

El sistema dispone de dos entradas de alimentación: una para tensión continua de 12 V, que representa la batería, y otra para tensión alterna, que simula las fuentes de línea, grupo electrógeno e inversor.

Asimismo, cuenta con un conjunto de luces testigo que actúan como carga real, en reemplazo de los dispositivos presentes en un motorhome.



FIGURA 4.7. Equipo Victron MultiPlus-II y módulo Cerbo GX.

En la pestaña principal se accionaron los pulsadores y se verificó la respuesta de las luces testigo asociadas. Asimismo, se aplicaron variaciones sobre los sensores de nivel de los tanques, con el fin de simular distintos estados y comprobar la actualización de los indicadores en la interfaz.

Este procedimiento se replicó en las demás pestañas, donde se evaluaron los distintos controles implementados y su efecto sobre los dispositivos físicos del sistema.

En relación con la comunicación con los equipos Victron, se utilizó la pestaña energía para controlar el equipo MultiPlus.

Al activar el Multiplus en modo inversor, el equipo comenzó a convertir la tensión de batería en una señal alterna de 220 V. Como carga, se conectó un soldador de tipo *cautín* de alta potencia, lo que permitió observar la variación de la potencia indicada en la aplicación.

Por otro lado, se conectó el Multiplus a la red domiciliaria y se lo configuró en modo cargador. Mediante el uso de una pinza amperométrica, se verificó que el valor de corriente de carga mostrado en la aplicación resultaba coherente con la medición real.

Como criterio de validación, se consideró satisfactorio que los estados de los actuadores coincidieran con las acciones realizadas desde la aplicación, y que las variables medidas presentaran valores coherentes con las lecturas obtenidas mediante instrumentos externos.

Los resultados obtenidos indicaron que el sistema respondió correctamente a las acciones de control, tanto en los dispositivos de la red de Enertik como en los equipos del ecosistema Victron. Las magnitudes eléctricas visualizadas en la aplicación resultaron consistentes con las mediciones realizadas.

Adicionalmente, se realizó una prueba en un motorhome real durante una visita a la fábrica, donde se evaluaron los controles en condiciones reales de operación. En esta instancia, el sistema presentó un comportamiento adecuado y consistente con los ensayos de laboratorio.



FIGURA 4.6. Equipos Enertik usados en los ensayos.

Para la comunicación con el ecosistema Victron, el sistema incluye un módulo Cerbo GX y un equipo MultiPlus-II 12/3000/120-32 230V [37], provisto por la empresa Enertik, capaz de operar en modo inversor y cargador, tal como se muestra en la figura 4.7.

Si bien el desarrollo del firmware se realizó sobre este modelo, los registros utilizados resultan compatibles con otros equipos de la misma línea de inversores/cargadores Victron.

En este entorno se ensayó la integración del microcontrolador ESP32 con los equipos reales de la red del motorhome.

Para ello, se instaló la aplicación en un teléfono celular con sistema operativo Android y se estableció la conexión a la red Wi-Fi generada por el ESP32.

A continuación, se procedió a evaluar el funcionamiento de cada una de las secciones de la aplicación.

En la pantalla home se pusieron a prueba los pulsadores y se verificó el accionamiento de las luces testigo correspondientes.

También se ejerció presión sobre los sensores de nivel de los tanques, con el objetivo de simular distintas condiciones de carga y analizar la respuesta de la aplicación.

No obstante, no fue posible validar la comunicación con equipos Victron en el vehículo, debido a la ausencia de este equipamiento en la unidad ensayada. Esta verificación queda planteada como trabajo futuro.

Finalmente, debido a restricciones de la empresa fabricante, no se dispuso de registros fotográficos de los ensayos realizados en el motorhome.



FIGURA 4.7. Equipo Victron MultiPlus-II 12/3000/120-32 230V y módulo Cerbo GX.

Este procedimiento se replicó en las demás pestañas, así como también se evaluaron cada una de las funcionalidades implementadas.

En cuanto a la comunicación con la red Victron, se utilizó la pestaña energía de la aplicación para controlar el equipo Multiplus.

Al activar el Multiplus en modo inversor, el equipo comenzó a convertir la tensión de batería en una señal alterna de 220 V. Como carga, se conectó un soldador de tipo *cautín* de alta potencia, lo que permitió observar la variación de la potencia indicada en la aplicación.

Por otro lado, se conectó el Multiplus a la red domiciliaria y se lo configuró en modo cargador. Mediante el uso de una pinza amperométrica, se verificó que el valor de corriente de carga mostrado en la aplicación resultaba coherente con la medición real.

En conjunto, estos ensayos arrojaron resultados favorables, donde el ESP32 demostró capacidad para controlar tanto los equipos de la red de Enertik como los dispositivos del ecosistema Victron.

Adicionalmente, se realizó una prueba en un motorhome real durante una visita a la fábrica. En dicha instancia, se evaluaron los distintos controles en condiciones reales de operación y se obtuvieron resultados satisfactorios.

Sin embargo, no fue posible ensayar la comunicación con equipos Victron en el vehículo, ya que al momento de la prueba no se disponía de una unidad equipada con este sistema. Por lo tanto, esta validación queda pendiente para trabajos futuros.

Finalmente, debido a políticas de la empresa fabricante de motorhomes, no fue posible registrar imágenes de los ensayos realizados durante la visita.